🏥 Hospital Management System

A Console-Based Hospital Operations Platform Built with Modern C++ & OOP

Simulates a real hospital environment where Admins, Receptionists, Doctors, and Accountants each operate through dedicated, role-based modules — with all data persisted locally via CSV files.

C++ OOP STL File Handling Visual Studio

📖 Overview

The **Hospital Management System** is a console-based application built in **C++** that automates day-to-day hospital operations. It provides dedicated modules for four types of users — **Admin, Receptionist, Doctor, and Accountant** — each with secure login and role-specific functionality.

All records (patients, doctors, staff, appointments, and billing history) are stored in **CSV files**, ensuring data persists between sessions without requiring a database server.

This project was built as a hands-on learning exercise to strengthen core programming fundamentals, object-oriented design, and file-based data management in C++, and comprises **1500+ lines of code**.

✨ Features

👨‍💼 Admin Module

Capability	Description
🔐 Secure Login	Authenticated admin access
➕ Add Staff	Add Receptionists, Doctors, and Accountants
➖ Remove Staff	Delete Receptionists, Doctors, and Accountants
🗑️ Delete Patients	Remove patient records
🛏️ View History	Access full hospital activity history

👩‍💼 Receptionist Module

Capability	Description
🔐 Login Authentication	Secure receptionist access
➕ Add New Patient	Register new patients into the system
👥 View All Patients	Browse complete patient records
👨‍⚕️ View Doctors	List all available doctors
📅 Create Appointment	Schedule patient-doctor appointments
❌ Cancel Appointment	Cancel existing appointments
📋 View Appointments	Review all scheduled appointments

👨‍⚕️ Doctor Module

Capability	Description
🔐 Secure Login	Authenticated doctor access
📅 View Assigned Appointments	See patients scheduled for the day
🩺 Examine Patients	Record examination details
💊 Write Prescriptions	Issue prescriptions to patients
🔄 Update Appointment Status	Mark appointments as completed/in-progress

💰 Accountant Module

Capability	Description
🔐 Login Authentication	Secure accountant access
🧾 Generate Bills	Create itemized patient bills
➗ Calculate Total Charges	Compute doctor fee + medicine cost
🗃️ Store Billing History	Maintain a complete billing ledger
💳 Manage Payment Records	Track patient payment status

👤 Patient & Appointment Management

- Add, delete, and search patient records
- Persistent patient data storage
- Create and cancel appointments
- Track appointment status and doctor assignment

🛏️ Billing & History

- Automated bill generation
- Doctor fee + medicine cost calculation
- Complete, persistent billing history

🛠️ Technologies Used

Category	Technology
Language	C++
Paradigm	Object-Oriented Programming (OOP)
Data Handling	File Handling (CSV)
Libraries	Standard Template Library (STL)
IDE	Visual Studio

💡 OOP Concepts Implemented

- 🏗️ Classes & Objects
- 🧬 Inheritance
- 🔒 Encapsulation
- 🎭 Abstraction
- 🤝 Friend Functions
- 🏭 Constructors
- [redacted]
- 📌 Static Member Functions

🗂️ Data Storage

All application data is persisted using **CSV files**, so information remains available across sessions without needing an external database.

File	Purpose
`Patients.csv`	Stores patient records
`Doctors.csv`	Stores doctor profiles
`Reception.csv`	Stores receptionist accounts
`Accountants.csv`	Stores accountant accounts

`Appointments.csv`	Stores appointment details
`History.csv`	Stores complete hospital activity history

💻 Source Files

File	Purpose
`main.cpp`	Application entry point
`hospital.h`	Hospital class declaration
`hospital.cpp`	Hospital class implementation
`person.h`	Person (base) class declaration
`person.cpp`	Person (base) class implementation

🗂️ Project Structure

```
Hospital-Management-System/
│
├── main.cpp
├── hospital.h
├── hospital.cpp
├── person.h
├── person.cpp
├── Patients.csv
├── Doctors.csv
├── Reception.csv
├── Accountants.csv
├── Appointments.csv
├── History.csv
└── README.md
```

🚀 Getting Started

1️⃣ Open in Visual Studio

Open the project folder as a project/solution in Visual Studio.

2️⃣ Build

Build the solution using Visual Studio's build tools.

3️⃣ Run

Execute the generated binary to launch the application.

🎯 Learning Outcomes

Through building this project, the following skills were practiced and strengthened:

- Large-scale C++ application design
- Object-Oriented Programming principles
- File handling and persistent storage
- STL containers and usage
- Building authentication systems
- Modular software architecture
- CRUD operations

👨‍💻 Author

Syed Muhammad Haleem Software Engineering Student at University of Sargodha Currently in 2nd Semester

Skills Completed: - 🌐 HTML & CSS - 🐍 Python - 🗄️ SQL - 💻 C++ - 🐘 OOP in Python and C++

Currently Learning: - 🧮 Data Structures & Algorithms in C++ - 🐘 PostgreSQL

⭐ Support

If you found this project useful or interesting, please consider giving it a **star** ⭐ on GitHub.

Feedback, suggestions, and contributions are always welcome!